DeepEP

Deep Dive into DeepSeek — Chapter 26

Yin Yang

Foundation Books Project

The one rule of this chapter

Picking the right expert is not the hard part.
Getting the token there and back, in order, is.

- Sparse routing selects experts; expert parallelism turns that selection into a destination rank.
- DeepEP moves each token row to the rank that owns its selected expert, then returns expert outputs to the *original* token order.
- Origin id, route slot, expert id, and weight must survive the round trip through dispatch, expert compute, and combine.

Goal: follow one four-rank token through dispatch, the handle, combine, V2's generated kernel chain, topology, and failure recovery — and see where each layer lives in the DeepEP source.

Roadmap

1. The round trip: dispatch, handle, combine

Roadmap

1. The round trip: dispatch, handle, combine
2. The V2 surface, resource selection, and overlap

Roadmap

1. The round trip: dispatch, handle, combine
2. The V2 surface, resource selection, and overlap
3. Topology, transport, and channel epochs

Roadmap

1. The round trip: dispatch, handle, combine
2. The V2 surface, resource selection, and overlap
3. Topology, transport, and channel epochs
4. Legacy V1 and the generation boundary

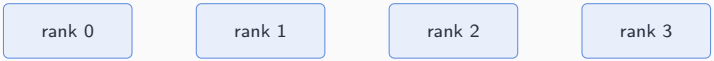
Roadmap

1. The round trip: dispatch, handle, combine
2. The V2 surface, resource selection, and overlap
3. Topology, transport, and channel epochs
4. Legacy V1 and the generation boundary
5. Reading a performance claim honestly

The round trip: dispatch, handle, combine

Four ranks, eight experts, one running example

owner$(e) = \lfloor e/2 \rfloor$, $g = sM_{\max} + i$ (origin id, recovered by combine)



- Rank 0 owns expert 1 locally, but still sends a route for expert 6 to rank 3 — ownership is per expert, not per rank.

Four ranks, eight experts, one running example

owner$(e) = \lfloor e/2 \rfloor$, $g = sM_{\max} + i$ (origin id, recovered by combine)

- Rank 0 owns expert 1 locally, but still sends a route for expert 6 to rank 3 — ownership is per expert, not per rank.

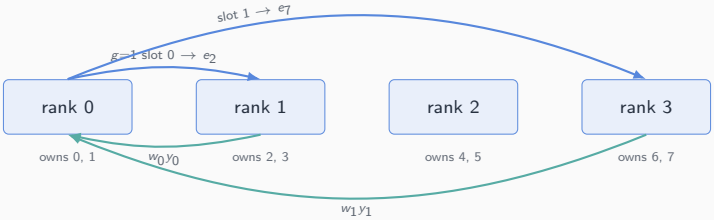
Four ranks, eight experts, one running example

owner$(e) = \lfloor e/2 \rfloor$, $g = sM_{\max} + i$ (origin id, recovered by combine)

- Rank 0 owns expert 1 locally, but still sends a route for expert 6 to rank 3 — ownership is per expert, not per rank.

Four ranks, eight experts, one running example

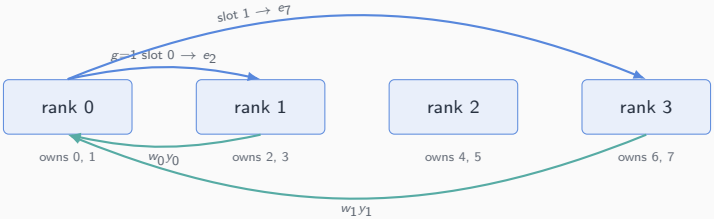
owner$(e) = \lfloor e/2 \rfloor$, $g = sM_{\max} + i$ (origin id, recovered by combine)



- Rank 0 owns expert 1 locally, but still sends a route for expert 6 to rank 3 — ownership is per expert, not per rank.
- Combine returns *both* contributions to token 1 on rank 0, still carrying their original weights.

Four ranks, eight experts, one running example

owner$(e) = \lfloor e/2 \rfloor$, $\quad g = sM_{\max} + i$ (origin id, recovered by combine)



- Rank 0 owns expert 1 locally, but still sends a route for expert 6 to rank 3 — ownership is per expert, not per rank.
- Combine returns *both* contributions to token 1 on rank 0, still carrying their original weights.

Swapped slots or a lost origin id can produce a plausible-looking value in the wrong row. DeepEP's job is to make that impossible.

Dispatch deduplicates by destination, not by expert

Token g	Experts $\rightarrow$ owners	Dispatch rows
0	$(1, 6) \rightarrow (0, 3)$	one row to rank 0, one to rank 3

Dispatch deduplicates by destination, not by expert

Token g	Experts $\rightarrow$ owners	Dispatch rows
0	$(1, 6) \rightarrow (0, 3)$	one row to rank 0, one to rank 3
1	$(2, 7) \rightarrow (1, 3)$	one row to rank 1, one to rank 3

Dispatch deduplicates by destination, not by expert

Token g	Experts $\rightarrow$ owners	Dispatch rows
0	$(1, 6) \rightarrow (0, 3)$	one row to rank 0, one to rank 3
1	$(2, 7) \rightarrow (1, 3)$	one row to rank 1, one to rank 3
2	$(4, 5) \rightarrow (2, 2)$	one row to rank 2, two valid local slots

Dispatch deduplicates by destination, not by expert

Token g	Experts $\rightarrow$ owners	Dispatch rows
0	$(1, 6) \rightarrow (0, 3)$	one row to rank 0, one to rank 3
1	$(2, 7) \rightarrow (1, 3)$	one row to rank 1, one to rank 3
2	$(4, 5) \rightarrow (2, 2)$	one row to rank 2, two valid local slots
3	$(0, 3) \rightarrow (0, 1)$	one row to rank 0, one to rank 1

Dispatch deduplicates by destination, not by expert

Token g	Experts $\rightarrow$ owners	Dispatch rows
0	$(1,6) \rightarrow (0,3)$	one row to rank 0, one to rank 3
1	$(2,7) \rightarrow (1,3)$	one row to rank 1, one to rank 3
2	$(4,5) \rightarrow (2,2)$	one row to rank 2, two valid local slots
3	$(0,3) \rightarrow (0,1)$	one row to rank 0, one to rank 1

Rank 0's per-destination split vector is $s^{(0)} = (2,2,1,2)$ — seven token rows instead of $4 \times 2 = 8$ route-slot copies.

Token 2's two experts share owner rank 2, so dispatch sends one payload row carrying two route slots, not two copies of the same activation.

The handle: communication layout is state

Definition (Dispatch handle)
The metadata returned by dispatch and consumed by combine to interpret receive buffers: origin-token rows, valid route slots, destination buffer locations, and per-expert receive layout.

- Rank 2 receives $g = 2$ (experts 4, 5) and $g = 7$ (expert 5), so its per-expert semantic counts are $M_4 = 1$, $M_5 = 2$.
- With alignment $A = 4$, physical prefix sums are $(4, 8)$ — padding reserves buffer space, not model tokens.

A cached handle is a layout promise: reuse is safe only while routing layout and ownership stay compatible with the call that produced it (Section 3).

Combine restores origin order, not receive order

Token 1 routes to expert 2 on rank 1 and expert 7 on rank 3, weights 0.65 and 0.35:

$$z_1 = 0.65\,(2, 0, 1, 1) + 0.35\,(0, 4, 1, 3) = (1.3,\ 1.4,\ 1.0,\ 1.7).$$

Combine restores origin order, not receive order

Token 1 routes to expert 2 on rank 1 and expert 7 on rank 3, weights 0.65 and 0.35:

$$z_1 = 0.65\,(2, 0, 1, 1) + 0.35\,(0, 4, 1, 3) = (1.3,\ 1.4,\ 1.0,\ 1.7).$$

- Swap the two route slots while keeping the weights, and the first coordinate becomes 0.7 instead of 1.3 — numerically plausible, semantically wrong.

Combine restores origin order, not receive order

Token 1 routes to expert 2 on rank 1 and expert 7 on rank 3, weights 0.65 and 0.35:

$$z_1 = 0.65\,(2, 0, 1, 1) + 0.35\,(0, 4, 1, 3) = (1.3,\ 1.4,\ 1.0,\ 1.7).$$

- Swap the two route slots while keeping the weights, and the first coordinate becomes 0.7 instead of 1.3 — numerically plausible, semantically wrong.
- The abstract rule: $z_g = b_g + \sum_{j=0}^{k_{\text{top}}-1} w_{g,j}\, y_{g,j}$, reduced in the same route-slot order dispatch recorded.

Combine restores origin order, not receive order

Token 1 routes to expert 2 on rank 1 and expert 7 on rank 3, weights 0.65 and 0.35:

$$z_1 = 0.65\,(2, 0, 1, 1) + 0.35\,(0, 4, 1, 3) = (1.3,\ 1.4,\ 1.0,\ 1.7).$$

- Swap the two route slots while keeping the weights, and the first coordinate becomes 0.7 instead of 1.3 — numerically plausible, semantically wrong.
- The abstract rule: $z_g = b_g + \sum_{j=0}^{k_{\text{top}}-1} w_{g,j}\, y_{g,j}$, reduced in the same route-slot order dispatch recorded.

Masked route slots contribute zero. An implementation that lets a masked slot carry stale memory into the sum breaks this invariant even when every valid slot was routed correctly.

The V2 surface, resource selection, and overlap

One `ElasticBuffer`, a generated kernel chain

Round-trip phase	Recorded runtime family
Notify and move	`DispatchRuntime`
Materialize receive layout	`DispatchCopyEpilogueRuntime`
Transform rows (outside DeepEP)	grouped expert GEMM
Move contributions	`CombineRuntime`
Restore origin rows	`CombineReduceEpilogueRuntime`
Fence communication state	`BarrierRuntime` (separate API)

"Notify" is a warp role inside dispatch movement, not a separate launch. Movement kernels carry their own embedded barriers; the explicit `BarrierRuntime` buffer fence is adjacent control state, not another token-row transform.

Resource choice is a traffic model, not a constant

$$S_{\text{raw}} = \max\left\{\frac{B_b}{\tau_b}\frac{m_r}{b_r}, \frac{B_b}{\tau_b}\frac{m_w}{b_w}\right\}, \qquad Q_{\text{direct}} = \min\{Q_{\text{alloc}}, S, 9\}$$

- $D(G, E, k)$ estimates expected distinct destination groups from top-k occupancy; the selector balances modeled HBM read/write demand against the bottleneck fabric.
- Worked teaching example (8 ranks, $E = 64$, $k = 8$, $B_{\text{NVL}} = 800$ GB/s): $S_{\text{raw}} \approx 18.29$, so $S = 24$ after headroom and even alignment, and direct mode picks $Q_{\text{direct}} = 9$.

[DERIVED: DeepEP V2 selector formulas, declared hypothetical inputs] — a transparent execution of the model, not a measured optimum. The README's reported 4–6 SM V3-like training figure is a separate repository claim (Section 6).

Event overlap: a timing relation, not a semantic change

Definition (Event overlap)
A stream event or equivalent dependency so communication can run concurrently with independent compute; the consuming stream waits only before it reads communication outputs.

- Decode can reuse a cached handle when the route layout is unchanged step to step — skipping layout recomputation and a CPU-exposing synchronization point.
- `EventOverlap` is also an ownership object: it must stay alive until *every* consuming stream, including CUDA-graph replay, has installed its wait.

Releasing the handle after only the first stream waits is a use-after-lifetime risk, not an optimization.

Topology, transport, and channel epochs

Scale-up vs. scale-out, direct vs. hybrid

	Direct	Hybrid
Path	straight to final owner	scale-out rail, then scale-up forward

Scale-up vs. scale-out, direct vs. hybrid

	Direct	Hybrid
Path	straight to final owner	scale-out rail, then scale-up forward
Ownership	one destination rank	an intermediate ingress rank too

Scale-up vs. scale-out, direct vs. hybrid

	Direct	Hybrid
Path	straight to final owner	scale-out rail, then scale-up forward
Ownership	one destination rank	an intermediate ingress rank too
Kernels	`dispatch_impl`/`combine_impl`	`hybrid_dispatch_impl`/`hybrid_combine_impl`

Scale-up vs. scale-out, direct vs. hybrid

	Direct	Hybrid
Path	straight to final owner	scale-out rail, then scale-up forward
Ownership	one destination rank	an intermediate ingress rank too
Kernels	`dispatch_impl`/`combine_impl`	`hybrid_dispatch_impl`/`hybrid_combine_impl`

Two-node placement of the running example (ranks 0,1 on node A; 2,3 on node B): token 0's route to expert 3 is scale-up; its route to expert 6 is scale-out.

Scale-up vs. scale-out, direct vs. hybrid

	Direct	Hybrid
Path	straight to final owner	scale-out rail, then scale-up forward
Ownership	one destination rank	an intermediate ingress rank too
Kernels	`dispatch_impl`/`combine_impl`	`hybrid_dispatch_impl`/`hybrid_combine_impl`

Two-node placement of the running example (ranks 0,1 on node A; 2,3 on node B): token 0's route to expert 3 is scale-up; its route to expert 6 is scale-out.

Same dispatch semantics, same origin id and combine restoration — topology changes the path, never the token-ownership rule.

From concept to code: the handle's field list

The dispatch-handle roles from Section 3 are exactly `EPHandle`'s constructor arguments.

from **DeepEP/deep_ep/buffers/elastic.py**

```
class EPHandle:
    """
    Communication handle returned by `ElasticBuffer.dispatch`.
    Can be reused as a cached handle ... to skip layout recomputation.
    """
    def __init__(self, do_expand, num_experts, expert_alignment,
                 num_max_tokens_per_rank, num_sms, topk_idx,
                 num_recv_tokens_per_expert_list,
                 psum_num_recv_tokens_per_scaleup_rank,
                 psum_num_recv_tokens_per_expert,
                 recv_src_metadata, dst_buffer_slot_idx, ...):
```

From concept to code: the handle's field list

The dispatch-handle roles from Section 3 are exactly `EPHandle`'s constructor arguments.

from **DeepEP/deep_ep/buffers/elastic.py**

```
class EPHandle:
    """
    Communication handle returned by `ElasticBuffer.dispatch`.
    Can be reused as a cached handle ... to skip layout recomputation.
    """
    def __init__(self, do_expand, num_experts, expert_alignment,
                 num_max_tokens_per_rank, num_sms, topk_idx,
                 num_recv_tokens_per_expert_list,
                 psum_num_recv_tokens_per_scaleup_rank,
                 psum_num_recv_tokens_per_expert,
                 recv_src_metadata, dst_buffer_slot_idx, ...):
```

`topk_idx` is the top-k view; `recv_src_metadata` and `dst_buffer_slot_idx` are the origin metadata and buffer slots; the two `psum_*` fields are the aligned prefix sums. [INSPECTED: DeepEP `deep_ep/buffers/elastic.py`]

A channel epoch has entry, progress, completion, failure

Definition (DeepEP channel epoch)
One use of a channel's payload slots, counters, tails, and finish state between the entry synchronization that makes them reusable and the exit synchronization that proves the data arrived.

- A timeout, trapped kernel, or communicator error leaves the epoch failed — the caller must not infer that slots or a cached handle are reusable.
- Direct mode's completion is one final barrier; hybrid mode adds a release-published tail that a forwarder must acquire before it may forward.

Recovery is fail-stop, not reconnect: rebuild the communicator, workspace, and stale handles before replay.

Legacy V1 and the generation boundary

V2 is not a rename of V1

Property	Legacy V1	Released V2
Backend	NVSHMEM normal/low-latency	NCCL Gin

V2 is not a rename of V1

Property	Legacy V1	Released V2
Backend	NVSHMEM normal/low-latency	NCCL Gin
Surface	`Buffer` (split methods)	one `ElasticBuffer`

V2 is not a rename of V1

Property	Legacy V1	Released V2
Backend	NVSHMEM normal/low-latency	NCCL Gin
Surface	`Buffer` (split methods)	one `ElasticBuffer`
Low-latency	zero-SM IBGDA receive hook	not preserved

V2 is not a rename of V1

Property	Legacy V1	Released V2
Backend	NVSHMEM normal/low-latency	NCCL Gin
Surface	`Buffer` (split methods)	one `ElasticBuffer`
Low-latency	zero-SM IBGDA receive hook	not preserved
Semantic invariant	same round trip	same round trip

V2 is not a rename of V1

Property	Legacy V1	Released V2
Backend	NVSHMEM normal/low-latency	NCCL Gin
Surface	`Buffer` (split methods)	one `ElasticBuffer`
Low-latency	zero-SM IBGDA receive hook	not preserved
Semantic invariant	same round trip	same round trip

The archived V1 low-latency path can progress RDMA without communication SMs via a receive hook. V2 does not preserve that zero-SM surface — a V1 SM-count observation cannot be transferred to a V2 run.

Reading a performance claim honestly

Skew turns balance into a communication problem

$$\rho_{\text{expert}} = \frac{\max_e M_e}{\frac{1}{E}\sum_e M_e}$$

- Eight balanced tokens give $\rho_{\text{expert}} = 1$ — yet cross-rank communication still exists even at perfect balance.

Skew turns balance into a communication problem

$$\rho_{\text{expert}} = \frac{\max_e M_e}{\frac{1}{E}\sum_e M_e}$$

- Eight balanced tokens give $\rho_{\text{expert}} = 1$ — yet cross-rank communication still exists even at perfect balance.
- Add four tokens routed to experts 6 and 7 (both on rank 3): $\rho_{\text{expert}} = 6/3 = 2$. Rank 3 now receives, and must return, a disproportionate share of the traffic.

Skew turns balance into a communication problem

$$\rho_{\text{expert}} = \frac{\max_e M_e}{\frac{1}{E}\sum_e M_e}$$

- Eight balanced tokens give $\rho_{\text{expert}} = 1$ — yet cross-rank communication still exists even at perfect balance.
- Add four tokens routed to experts 6 and 7 (both on rank 3): $\rho_{\text{expert}} = 6/3 = 2$. Rank 3 now receives, and must return, a disproportionate share of the traffic.

A hot-rank histogram is a warning signal, not a bubble proof — a communication bubble claim needs a wait interval in an actual trace.

Reported numbers keep their evidence stamp

Topology	NIC	Dispatch	SM budget
EP 8×2	CX7	90 GB/s RDMA	12

Reported numbers keep their evidence stamp

Topology	NIC	Dispatch	SM budget
EP 8×2	CX7	90 GB/s RDMA	12
EP 8 (NVLink, peak)	—	726 GB/s	64

Reported numbers keep their evidence stamp

Topology	NIC	Dispatch	SM budget
EP 8×2	CX7	90 GB/s RDMA	12
EP 8 (NVLink, peak)	—	726 GB/s	64
EP 8 (NVLink, min SM)	—	643 GB/s	24

[REPORTED: DeepEP V2 README, commit 60d44037]

These are logical bandwidth from the released README under its own V3-shaped workload — calibration for the released transport, not baselines for a different route shape or a different measurement.

Handoffs, failures, and what comes next

- The expert-parallel handoff record — origin id, experts, weights, owner ranks, destination slots, scale metadata, reduction policy — must survive router $\rightarrow$ dispatch $\rightarrow$ expert compute $\rightarrow$ combine $\rightarrow$ runtime.
- Dangerous failures are metadata failures: wrong owner map, a stale cached handle, padding leaking into combine, a masked slot carrying stale memory — each can preserve tensor shape while changing meaning.
- JIT lifecycle adds its own failure class: incomplete cache directories, same-identity publication races, cross-rank identity drift.

DeepGEMM begins after rows are local; EPLB changes placement, not transport; Mega MoE fuses the whole path under a different implementation without ever calling DeepEP.

Concept-to-code map for the chapter

- **handle and buffer layout** → `DeepEP/deep_ep/buffers/elastic.py`
- **reference dispatch/combine semantics** → `DeepEP/deep_ep/utils/refs.py`
- **JIT compile, cache, and launch** → `DeepEP/csrc/jit/compiler.hpp`
- **legacy V1 IBGDA transport** → `DeepEP/csrc/kernels/legacy/ibgda_device.cuh`

Move the row, keep the identity, restore the order.

Next: EPLB turns the receive counters this chapter exposes into a placement decision, and Mega MoE asks whether the whole round trip can be fused instead of dispatched.